Can ProxyExec & sched_ext get along?

John Stultz <jstultz@google.com>
Suleiman Souhlal <suleiman@google.com>



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Previously...

OSPM25: Video

LPC2024: Video

OSPM24: Video

LPC2023: Video

OSPM23: Video

OSPM19: Video (Juri Lelli's talk)



Why ProxyExec?

Currently

Android Suffers from Priority Inversion

Android conceptually has **FOREGROUND** and **BACKGROUND** tasks.

Want to avoid **BACKGROUND** tasks impacting **FOREGROUND** task performance.

If we use any type of restriction or throttling of **BACKGROUND** tasks, we open ourselves up to priority inversion, where **BACKGROUND** tasks take kernel locks, and then are preempted. Important **FOREGROUND** tasks then have to wait until **BACKGROUND** task can run again to release the lock. This is a common cause of UI jank.

Goal

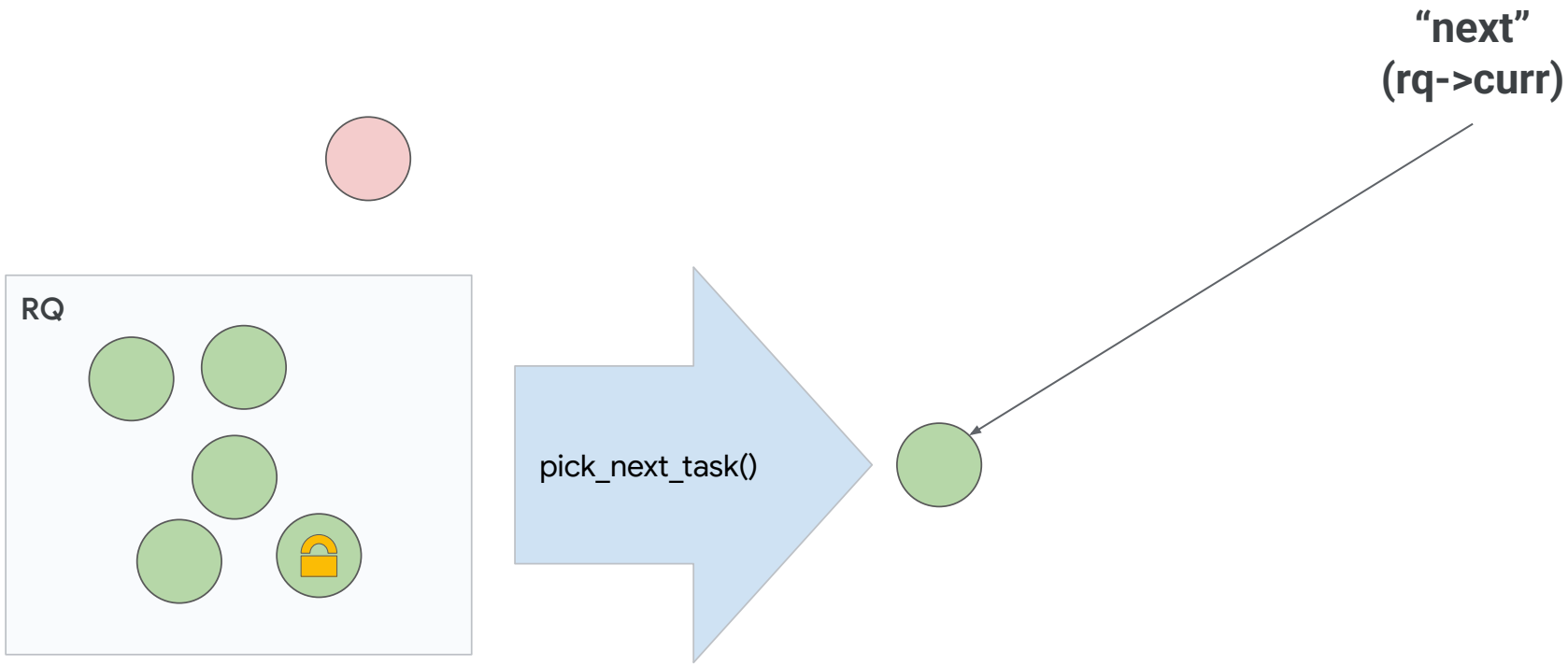
ProxyExec solves mutex priority inversion

Generalized form of priority inheritance.

FOREGROUND tasks, when waiting on a lock held by BACKGROUND tasks will “boost” the BACKGROUND tasks, allowing them to run until they release the needed lock.

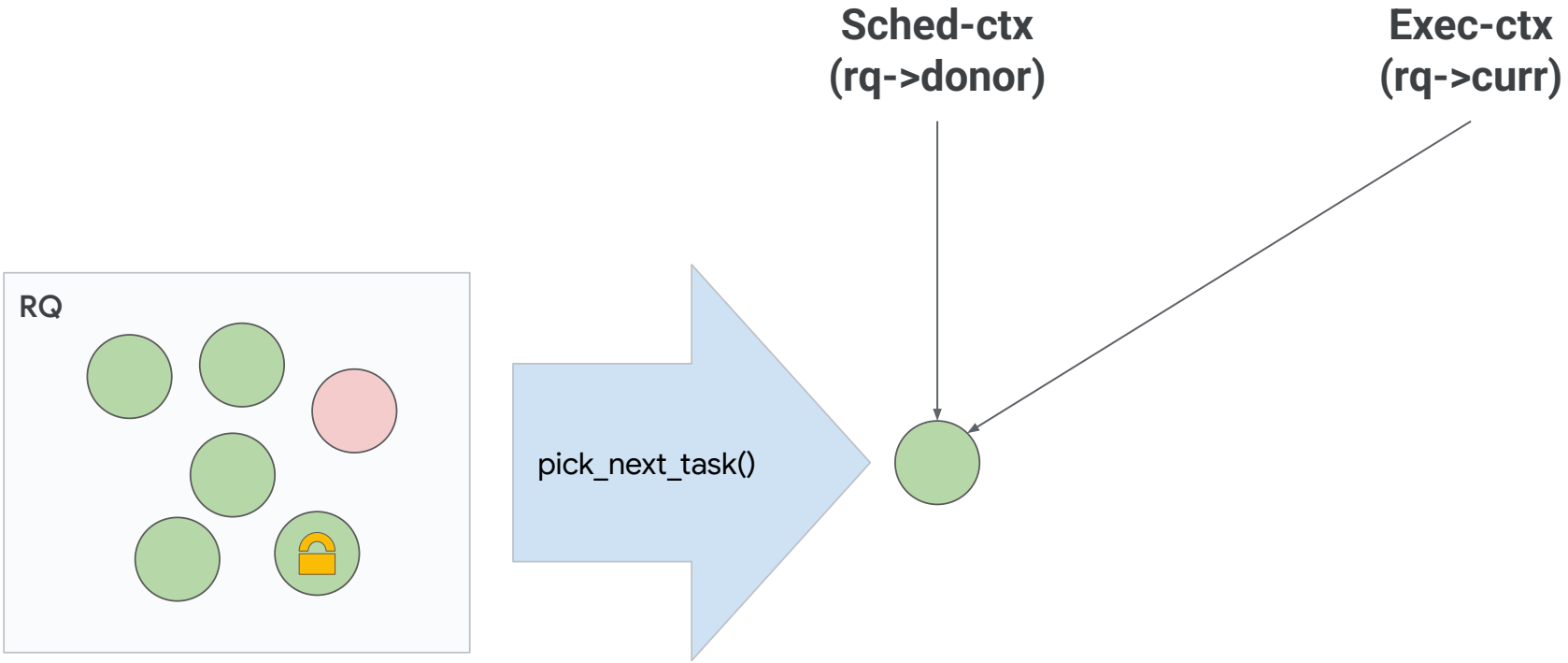
Allows utilization of various ways of restricting BACKGROUND tasks (cpu & cpuset cgroups, SCHED_IDLE, etc) so that that they don’t impact FOREGROUND tasks.

Without ProxyExec:



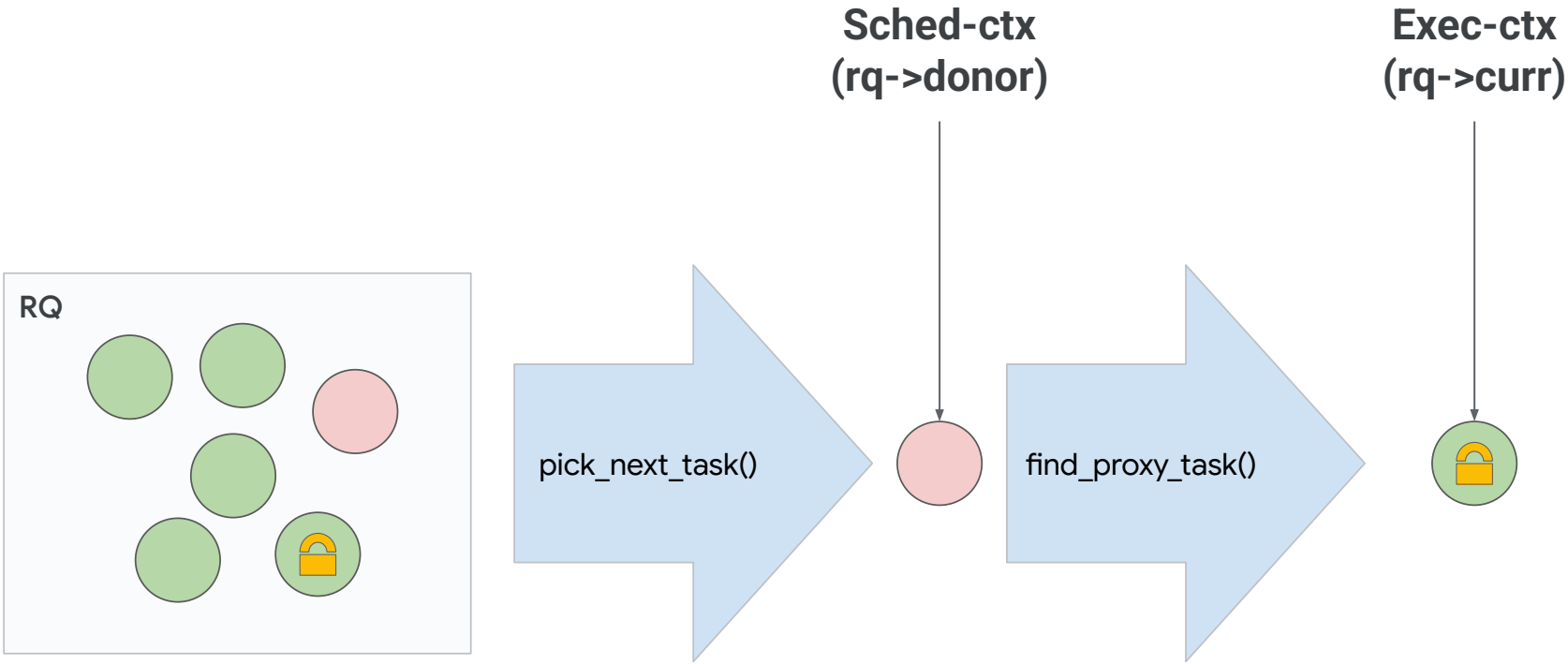
Google

ProxyExec Concept: Split Contexts



Google

ProxyExec Concept: Split Contexts



Benefits

Avoids PI Caused Latency

Example case of **foreground** filesystem operation latencies, with background tasks doing the same workload.

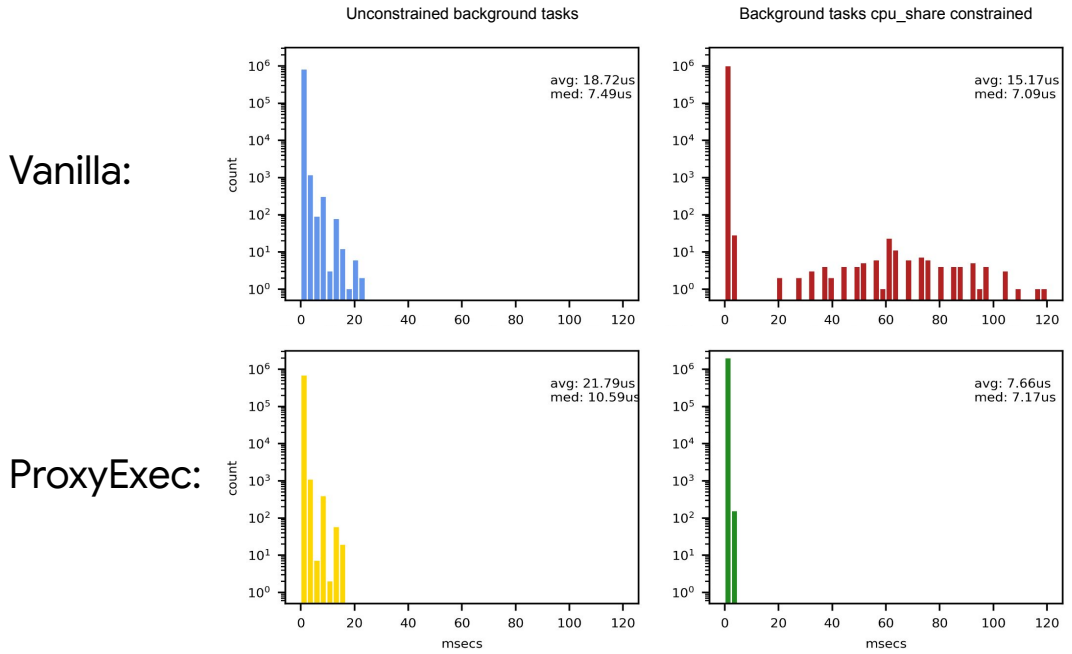
Chart shows histogram of measured latencies.

On the left, background tasks are running unconstrained, and on the right, we are using cpu_share cgroups to constrain background tasks.

Y-axis is log-scale.

X-axis is in ***milliseconds***

Histogram of time to move a file between directories with background tasks doing the same



https://github.com/johnstultz-work/priority-inversion-demo

Upstream Status & What's New

6.13: Prep patches(v12)

- Wakeups out of mutex::wait_lock ctx
- Helper Functions
- Split sched/exec contexts

Donor Migration (v24!)

- New task::blocked_lock
- task::blocked_on PROXY_WAKING
- Proxy-migration & Return-migration

Chain migration

- Handle RT/DL balancing at the chain-level

6.17: Single RQ Proxy(v18)

- Added SCHED_PROXY_EXEC config option
- Reworked task::blocked_on
- Initial chain-walk on single rq
- Dequeue donor on anything else

Sleeping owner enqueuing

- Complex sleeping owner enquing and activation logic

rwsem & sched_ext

- Sulieman enabled proxy-rwsems!
- Fixes for split contexts in ext.c logic
- Removing Kconfig exclusivity between SCHED_CLASS_EXT & SCHED_PROXY_EXEC



Quick Tangent:

Android Vendor Hooks

- Android's mechanism for vendors to modify the behavior of Generic Kernel Image (GKI).
- Similar to sched_ext, as it allows for deep customization of kernel behavior.
- Very powerful (can hook almost anywhere and replace any logic), but obviously, not upstream friendly!



ProxyExec in Android

- **Full series merged in android16-6.12**, but disabled by default (can be enabled via boot argument), so vendors could update Vendor Hooks that wouldn't expect the split contexts in ProxyExec
- **Vendors/OEMs evaluating it** - I've been involved helping fix Vendor Hooks and devices are using it in dogfood. Seeing additional vendors also start to play with it.

sched_ext

Scheduler decisions made via BPF programs!

Custom bpf schedulers!

Landed upstream right before early ProxyExec work. Didn't need to pay attention to the coming split-context changes in ProxyExec.

From experience with Vendor Hooks, I expressed some concern that core scheduling logic choosing to run a different task then what was selected might cause some problems with sched_ext.

Peter suggested make it build-time exclusive, and let sched_ext folks figure it out!

At the time, Android didn't seem likely to adopt sched_ext, as it overlapped with Android Vendor Hooks

SEP: Somebody Else's Problem!



sched_ext in Android

- Made **available in android16-6.12** for OEMs but **unsupported** by Google.
- **Vendors/OEMs evaluating it** - though we are seeing some interesting Vendor Hooks being added to further modify sched_ext behavior in the kernel.

ProxyExec and sched_ext are on a likely collision course, with folks wanting to use both together soon!

What to do?

Have to get things working together!

Concern: With ProxyExec there can be two “current” tasks

1) Execution context (proxy/rq->curr)
2) Scheduler context (rq->donor)

Usually they are the same.

In the case of priority inversion, where a lock-waiter is selected (for donor), we will pick the lock-owner to actually run(curr).

Breaks some common assumptions:

- Task that is selected by pick_next_task(), may not actually run
- prev != current, prev == rq->donor
- Accounting **mostly** done against rq->donor (vruntime, etc)
- But cputime charged to actually running task (keep top output sane)
- rq->donor can change during rq-lock drop! (prev isn’t stable across lock drop!)

For sched_ext, what is the uAPI commitment to bpf schedulers?

- If bpf chooses a task, what happens if that task doesn’t actually run?
- Could the bpf code get confused?
- Could this behavior change effectively break kernel uAPI?

Testing: ProxyExec + sched_ext

Removed Kconfig exclusions, and started testing:

- Mostly started with scx_simple
- Initially, very unstable.
  - locktorture and other loads would quickly trip hung task warnings and rcu stall errors
  - Ended up unrelated to ProxyExec! Just RT starvation!
  - Had to pull in the scx-dl_server branch
  - Stability much improved! (at least for scx_simple!)
- sched_ext kselftests!

Few issues found:

- SCX_OPS_ENQ_LAST confused by proxy_resched_idle()
- task_can_run_on_remote_rq() hitting is_migration_disabled() causing scx_central to bail

Sched_ext changes made:

- Merged in scx-dl_server
- Change rq->curr to rq->donor where appropriate
- Checks to avoid migrating rq->donor task or blocked tasks
- Fix for handling null next in put_prev_task()

ProxyExec changes made:

- Lock-drop in sched_ext clearly illustrated ‘prev’ passed to pick_next_task() is unstable across rq-lock drops w/ proxy.
- Needed to rework how ‘prev’ is handled in core and class schedulers to avoid it going stale (sched_ext may need further work).

Testing issue:

The priority-inversion-demo test is helpful to validate ProxyExec correctness. It uses cgroups to limit background tasks in order to contrive priority inversion.

Most sched_ext schedulers ignore cgroups! So it's difficult to measure benefit when inversions don't happen frequently.

Needed to implement trivial bpf scheduler to contrive its own inversions!

Have to contrive priority-inversion in order to validate ProxyExec working

HACK ALERT!

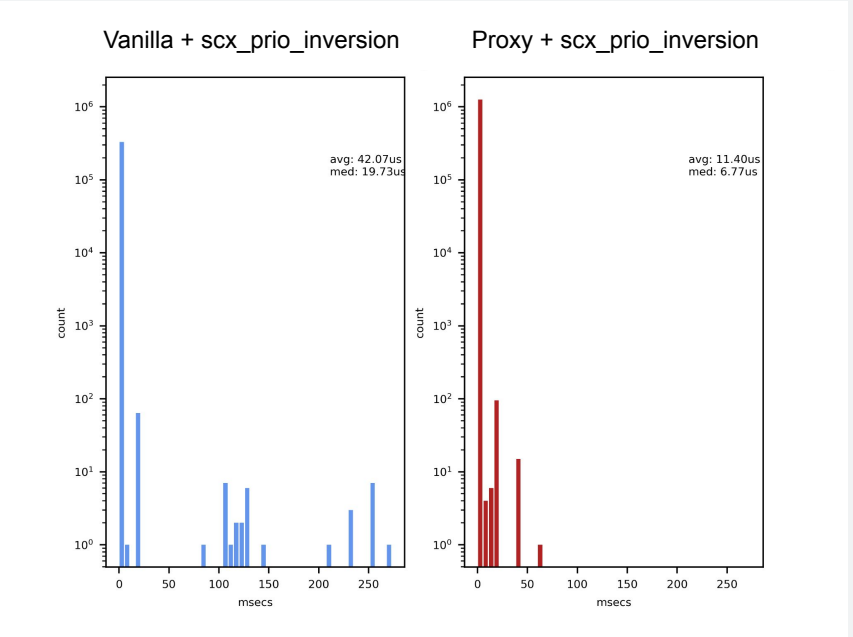
scx_prio_inversion

Modified scx_simple

Uses task->comm. Bumps up the “foreground” task and sandbags the “background” task in the DSQ:

```
if (!bpf_strncmp(p->comm, 10, "foreground")) {
        stat_inc(0);
        vtime -= 2*SCX_SLICE_DFL;
}
if (!bpf_strncmp(p->comm, 10, "background")) {
        stat_inc(1);
        vtime += 2*SCX_SLICE_DFL;
}
scx_bpf_dsq_insert_vtime(p, SHARED_DSQ, SCX_SLICE_DFL, vtime,
                         enq_flags);
```

Since SCX_SLICE_DFL defined 20ms default timeslices, you can see the 20ms “peaks” in the Proxy case.



Discussion: Are we good?

Wanted to ensure the DSQ interface the bpf schedulers use to interact with the EXT class scheduler is abstract enough that Proxy's split contexts don't confuse things. The ext.c logic seems like it can override bpf schedulers as well - so this all looks good.

Question: Are there any promises to bpf logic that the task they select will run?

ProxyExec + sched_ext appears to work ok in my testing. But I'm no sched_ext expert.

Discussion:

What else should I be testing for correctness?

Discussion:

How can you help? :)
Would love more folks playing and testing with ProxyExec!

SEP -> TCBYPT!

Discussion:

Other concerns?

Thank you

Particularly to folks who worked on the patch series before me:

Watkins, Straub, Niehaus, Peter Zijlstra, Juri Lelli, Valentin Schneider, Connor O'Brien

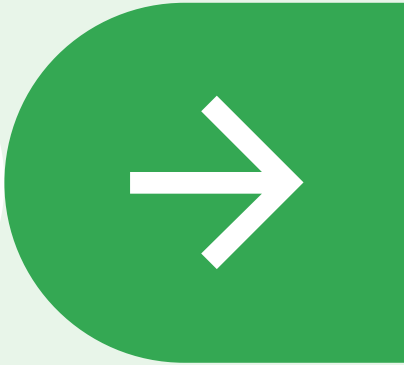
Also for their review and feedback & contributions:

Joel Fernandes, Qais Yousef, Metin Kaya, K Prateek Nayak, Steven Rostedt, Suleiman Souhlal and others!



Get in touch

jstultz@google.com
suleiman@google.com



Backup Slides

Plans for next year

Upstream!

Continue merging ProxyExec 😫

Try to keep momentum going and continue reworking and pushing ProxyExec upstream in digestible chunks that that can be well tested over time in the community.

Vendor Stuff

Push for wider use

Support vendors enabling ProxyExec on devices.

Also support vendors using sched_ext on devices with ProxyExec.

Resolve any bugs or issues as they are found.

MOOORE PROXY

Expand to even more lock types

Want to expand to proxy for other locking primitives (beyond mutex and rwsem)

- Binder (see Carlos' talk)
- Replacing rt-mutex (have to be a little more careful about lock handoff order)
- pi-futex
- Potentially extending normal futexes to track ownership, so futex_wait can proxy